

AI コードレビュー・オーケストレーション構成のコスト-欠陥検出性能比較

Cost-vs-Defect Detection Performance Comparison of AI Code Review Orchestration Configurations

高原 大明^{1*} 岡田 将吾¹ 伊集院 幸輝¹
Hiroaki Takahara¹ Shogo Okada¹ Koki Ijuin¹

¹ 北陸先端科学技術大学院大学

¹ Japan Advanced Institute of Science and Technology

Abstract: Aspect-parallel review, adversarial critique, and multi-run aggregation extend LLM code review beyond single-reviewer setups. Prior evaluations do not control compute budget, so we cannot tell whether these constructions add anything the raw compute would not. We build a defect-detection benchmark by inverting real bug-fix commits from four single-language open-source (OSS) repositories (Python, TypeScript, Go, Rust). One repository postdates current model cutoffs, giving a leakage-free control. Across 30 items, 12 models, 3 vendors, and 4 orchestrations, we rank by F_2 (Recall weighted $4\times$ over Precision to match the asymmetric cost of missed defects). 9 of 12 models (none of their 4 configurations enters the front) fall off the cost- F_2 Pareto front. Multi-run aggregation wins in 8 of 12 models and every one of the four repositories. Across cost bands, cheap models running Multi-run beat premium single-shot selection above \$0.01/item; returns flatten above \$0.05.

1 はじめに

汎用人工知能 (AGI; Artificial General Intelligence) の実現には、多様な課題に対して限られた計算資源の下で柔軟かつ効率的に推論を行う能力が求められる。近年、大規模言語モデル (LLM) はソフトウェア開発をはじめとする幅広い知的タスクで利用されており、コードレビューの自動化も研究・商用の両面で普及が進んでいる。LLM を用いたコードレビューの手法として、単一レビューで全観点を確認する構成が広く使われてきた。近年はさらに、観点別並列 (Parallel)、反証型構成 (Adversarial)、反復実行集約 (Multi-run) などオーケストレーション構成の提案が続いている [1]。しかし、これらの構成の優位性を報告する評価の多くは計算量 (トークン消費) を統制しておらず、観測された性能差が構成の構造に由来するのか、単なる計算量増加に由来するのかは切り分けられない。一般推論ドメインでは、予算を揃えると multi-agent debate 等の優位が消失・逆転するとの報告が複数ある [2]。一方、コードレビューでは AI の誤指摘はレビューが却下すれば対処できるのに対し、見逃しの発見には人手コードレビュー

を要する。この非対称性の下で上記の知見が成立するかは未検証である。

本研究は次の 3 つの研究課題を設定する。

- **RQ1:** 同等コスト水準で見たとき、オーケストレーション構成の間に欠陥検出率・偽陽性率の差があるか。
- **RQ2:** 観点別並列構成の優位性 (があるとするば) は、同等コストの Multi-run 集約 (純粋な計算量増加) と比較しても残るか。
- **RQ3:** 欠陥の種別によって有利な構成は異なるか。

本稿の貢献は次の 3 点である。

- 実オープンソースソフトウェア (以下 OSS) のバグ修正コミット逆適用による、計算予算を統制できる欠陥検出ベンチマークの構築 (§3)。
- 12 モデル $\times$ 4 構成 $\times$ 30 件のグリッドで得た (コスト, F_2) の実測 (§4-5)。
- 予算帯に依存する構成の優劣関係を示すパレートフロントの提示 (§5)。

*連絡先: 北陸先端科学技術大学院大学
E-mail: hiroaki@jaist.ac.jp

2 関連研究

Multi-agent コードレビュー. 役割分担型のレビューエージェント [1] をはじめ, LLM multi-agent 構成による欠陥検出の提案は多く, 分野の整理は近年の survey[3] が詳しい. これら既存研究は個別構成の効果を報告するが, コスト-性能面で複数構成を横断比較したものは調査の範囲では存在しない. 一方, 商用レビューエージェントの実証評価では, 多数のツールで指摘のシグナル比が低くノイズ過多であることが報告されており, 適合率を含めて評価する必要がある.

計算量統制下の multi-agent 評価. 一般推論タスクでは, multi-agent debate[2] や self-refinement[4] の性能向上が, 同一計算量の単純な多数決や複数候補からの最良選択と比較すると消失するとの報告がある. Wang らは強い prompt を与えた single-agent が multi-agent 議論のほぼ最良と同等となることを実証し, multi-agent 優位の境界を再考している [5]. コードレビュードメインでこの統制比較を行った研究は, 調査の範囲では存在しない.

欠陥ベンチマーク. Defects4J[6] や SWE-bench[7] など実バグ由来のベンチマークが広く使われているが, いずれも修正パッチ生成の評価が主目的であり, レビュー (差分に対する指摘) の評価を目的とした, 計算予算を統制できる形式のベンチマークは整備されていない (ソフトウェア工学 (software engineering; SE) における LLM 応用の広範な整理は [8] を参照). また, これら既知ベンチマークは LLM の学習データに含まれる可能性が高く, 暗記による性能の過大評価 (leakage) が指摘されている.

3 ベンチマーク構築

3.1 設計方針

実 OSS のバグ修正コミット F を逆適用し, 「バグを含む変更を提出する pull request (以下 PR)」を合成する. 基底状態を F 適用後の正常コードとし, F^{-1} をレビュー対象の差分として提示する. たとえば fix コミットが境界チェック `if (x < 0) return -1;` を追加していた場合, 逆適用ではこの行を削除した差分を PR として提示し, レビューがこの見落としを指摘できるかを評価する. これにより全リポジトリ・全バグに一律の手続きを適用でき, 正解 (ground truth) も fix の変更箇所として一意に定義される. 合成 PR であって実在した PR ではない点は本手法の限界であり, 6 節で議論する.

表 1: 対象リポジトリ.

リポジトリ	言語	ドメイン	ライセンス
fastapi	Python	Web FW	MIT
valibot	TypeScript	Schema 検証	MIT
chi	Go	HTTP ルータ	MIT
pgrust	Rust	DB エンジン	AGPL-3.0

3.2 対象リポジトリ

表 1 の 4 リポジトリを対象とする. リポジトリ効果と言語効果を分離するため, 各リポジトリが単一言語で構成されるものを選定し, 言語 (Python, TypeScript, Go, Rust) とドメイン (Web フレームワーク, スキーマ検証, HTTP ルータ, データベースエンジン) を分散させた. また既存ベンチマーク (SWE-bench 等) との重複がないことを確認し, leakage リスクを下げた.

pgrust は PostgreSQL の Rust 移植であり, 全履歴が 2026 年 6 月以降, すなわち本稿対象モデルの知識カットオフ後に作られた AI 支援開発リポジトリである. このため本稿対象モデルに対する leakage リスクを大幅に低減できる統制群として機能し, AI 支援下で作成されたコードを対象とするレビュー課題を含む. さらに PostgreSQL の回帰テスト群 (46,000 クエリ超) を欠陥実在性の判定基準として利用できる. 一方で単独作者・AI 支援によるバグ分布の偏りは 6 節で明記する.

3.3 構築パイプライン

構築は 4 段階からなる. 各段階の生成物は JSON としてリポジトリ²で公開している.

(1) **抽出.** リポジトリごとの commit 規約に合わせた正規表現で bug fix 候補コミットを抽出した (計 1,544 件).

(2) **機械フィルタ.** 非テストコードの変更が 100 行以下かつ 5 ファイル以下, docs・typo・revert 等を除外する条件で 816 件に絞った. 行数はテストコードを除いて数える (1 行の修正に大規模なテストが付随するケースを誤って除外しないため).

(3) **LLM 分類と懐疑的再検証.** API コスト削減のため 816 件全数投入は避け, post-cutoff 全数と目標未達リポジトリの pre-cutoff 補充を反復投入して累計 201 件を分類した. LLM エージェントに各候補の diff を読ませ, 実挙動バグの修正であるか (機能追加・リファクタ・移植作業の補完等を除外), 逆適用が自然なレビュー課題となるか, バグ種別, 深刻度を判定した. バッチ間

¹fastapi: <https://github.com/fastapi/fastapi> commit afe4112; valibot: <https://github.com/open-circle/valibot> commit 32247b3; chi: <https://github.com/go-chi/chi> commit 8b258c7; pgrust: <https://github.com/malisper/pgrust> commit c9aecbd.

²<https://github.com/hiroaki222/review-orchestration-bench>

で LLM の判定基準は一貫せず、判定が極端なバッチには反証プロンプトの再検証エージェントを適用し、4 件の判定を反転させた。最終的に 201 件中 135 件が逆適用可能 (真のバグ修正かつ逆適用が自然) と判定された (目標 85 件に対して $\sim 1.6\times$ の余裕)。分類は Claude 系 (Sonnet 5) で行い、異系統 LLM (Codex/GPT 系) による 17 件 (repo 比例層別) の独立再判定で 12 件一致を確認した。LLM 分類に由来する選定バイアスの議論は 6 節に譲る。

(4) **層別サンプリング**. 乱数を用いない決定的アルゴリズム (知識カットオフ後のコミットを優先し、残りをバグ種別のラウンドロビンで補充) により、85 件 (fastapi 25 / valibot 20 / chi 15 / pgrust 25) を選定した。内訳はカットオフ後 49 件・カットオフ前 36 件であり、種別は logic (論理誤り) 34, type-safety (型安全性) 15, error-handling (例外処理) 10, concurrency (並行性) 7, security (セキュリティ) 6, その他 13 である。

3.4 Leakage 統制

境界日を 2026-01-01 に設定し、以降のコミットを post-cutoff として優先採用、不足分は pre-cutoff から補充した上で、前後の別を共変量として分析に組み込む。この境界は主要モデルのカットオフのうち下限に近い日付を選び、全モデルで知識カットオフ後とみなせる範囲を最大化した。モデルごとの真のカットオフは異なるため厳密には近似だが、pgrust は全履歴が 2026-06 以降のため全モデル共通のカットオフ後データとなる。事前調査では、post-cutoff のみで必要数を確保できるリポジトリは pgrust のみであり (他 3 リポジトリの post-cutoff における逆適用可能件数は 4~11 件)、時期混合と共変量統制は必須であった。

4 実験設計

構築したベンチマークから 30 件を層別抽出し、次の 4 構成 $\times$ 12 モデル $\times$ 30 件のグリッドで欠陥検出性能とコストを実測した。括弧内の数字は LLM 呼び出し回数を表す。

- **Single**: 単一レビューが全観点を担当 (1 回)。
- **Parallel**: 4 観点 (correctness, concurrency, api-misuse, security) を独立実行し和集合を取得 (4 回)。
- **Adversarial**: レビューが指摘を生成する初回 1 回の後、各指摘に反証プロンプトを 2 回適用し半数以上の支持で採用する (指摘数 k に対し $1+2k$ 回)。

- **Multi-run**: 同一プロンプトを 5 回実行し 2 票以上の指摘のみ採用する計算量統制の対照群 (5 回)。

全構成でシステムプロンプトは共通の基底 “*You are a senior software engineer reviewing a pull request for defects the change would introduce (not stylistic nitpicks).*” を用い、各指摘を {file, line_start, line_end, severity, category, summary} の JSON 配列として出力させる。Parallel では 4 観点 (correctness, concurrency, api-misuse, security) それぞれに焦点範囲を絞る補助文を前置し、Adversarial の反証プロンプトは “*Default to REFUTED unless the diff clearly demonstrates the bug.*” を用いる。Multi-run は Single と同一プロンプトを 5 回反復する。完全な文言は公開リポジトリの src/harness/prompts.py を参照。

モデルは Anthropic (Haiku 4.5, Sonnet 5, Opus 5, Fable 5), OpenAI (GPT-5 mini, GPT-5, GPT-5.6 luna, GPT-5.6 terra, o3), Google (Gemini 3.5 flash-lite, 3.6 flash, 3.1 pro-preview) の 3 ベンダ 12 モデルを対象とし、各ベンダで廉価モデルから高性能モデルまでの価格帯を横断させた。検出判定は指摘の (file, line 範囲) と ground truth のハンク範囲の重なり (許容 ± 2 行) を機械判定する。主指標には F_2 スコア ($\beta = 2$, $F_2 = 5PR/(4P + R)$) を採用する。これは再現率 (Recall; 真のバグを取りこぼさない度合い) を適合率 (Precision; 指摘が本物である割合) の 4 倍重視する指標である。コードレビュー実務において、AI が挙げた候補の却下はレビューの確認で対処できるが、AI が見逃したバグの発見は最終的に人手コードレビューを要する。前者に比べ後者の対応工数が高いため、見逃しを減らすことを優先する F_2 を主指標に据えた。参考として適合率と再現率も併記する。

5 結果

30 件 $\times$ 12 モデル $\times$ 4 構成 = 1440 件の結果を表 2 に示す (パレートフロント 6 点は P で明示)。コスト・ F_2 散布は図 1 に示す。

GPT-5 mini の Multi-run 構成が最良で $F_2 = 0.826$, 項目あたり平均 \$0.0206 を要した。項目あたりコストは最安 \$0.0005 から最高 \$0.2182 まで 415 倍の広がりを持つが、 F_2 はコストに単調ではない。構成別平均 F_2 は Multi-run が 0.602 と最も高く、続いて Single 0.536, Parallel 0.514, Adversarial 0.479 となる。再現率重視の F_2 では、Adversarial の反証プロンプトが誤検出を減らして適合率を上げる一方で本物のバグまで却下し再現率を下げるため、総合では Single を下回る。Multi-run の反復実行集約 (2/5 しきい値) は適合率と再現率の双方を伸ばし他構成を上回った。モデル別に最良 F_2 の構成を集計すると、12 モデル

表 2: 構成別の検出性能とコスト (各セル 30 件平均, $N = 30$). 太字はモデル内最高 F_2 , 下線付き太字は全体最高, P はパレートフロント (図 1 と対応). 略号: Sin = Single, Par = Parallel, Adv = Adversarial, MR = Multi-run.

Model	Cfg	Prec	Rec	F_2	\$/item
<i>Anthropic</i>					
Haiku 4.5	Sin	0.80	0.60	0.635 P	0.0019
	Par	0.25	0.68	0.503	0.0076
	Adv	0.86	0.47	0.518	0.0066
	MR	0.79	0.65	0.671P	0.0096
Sonnet 5	Sin	0.88	0.51	0.561	0.0091
	Par	0.33	0.62	0.526	0.0276
	Adv	0.90	0.38	0.432	0.0204
	MR	0.81	0.57	0.609	0.0402
Opus 5	Sin	0.82	0.72	0.738	0.0341
	Par	0.25	0.74	0.533	0.1215
	Adv	0.84	0.53	0.571	0.1243
	MR	0.80	0.72	0.736	0.1755
Fable 5	Sin	0.70	0.56	0.583	0.0584
	Par	0.29	0.62	0.502	0.1747
	Adv	0.85	0.41	0.459	0.1461
	MR	0.85	0.59	0.627	0.2182
<i>OpenAI</i>					
GPT-5 mini	Sin	0.59	0.29	0.327	0.0035
	Par	0.25	0.68	0.503	0.0143
	Adv	0.74	0.74	0.735 P	0.0102
	MR	0.84	0.82	0.826P	0.0206
GPT-5	Sin	0.88	0.53	0.575	0.0341
	Par	0.36	0.57	0.513	0.1036
	Adv	0.91	0.43	0.477	0.0682
	MR	0.65	0.53	0.550	0.1794
GPT-5.6 luna	Sin	0.97	0.49	0.539	0.0067
	Par	0.38	0.57	0.521	0.0234
	Adv	0.91	0.44	0.492	0.0119
	MR	0.82	0.53	0.570	0.0378
GPT-5.6 terra	Sin	0.97	0.50	0.554	0.0161
	Par	0.33	0.54	0.482	0.0464
	Adv	0.97	0.44	0.495	0.0238
	MR	0.86	0.54	0.587	0.0796
o3	Sin	0.94	0.47	0.523	0.0153
	Par	0.33	0.63	0.532	0.0487
	Adv	0.95	0.29	0.341	0.0357
	MR	0.86	0.56	0.601	0.0728
<i>Google</i>					
Gemini 3.5 flash-lite	Sin	0.96	0.38	0.435 P	0.0005
	Par	0.45	0.53	0.511P	0.0019
	Adv	0.94	0.24	0.277	0.0011
	MR	0.88	0.34	0.386	0.0024
Gemini 3.6 flash	Sin	0.93	0.40	0.449	0.0224
	Par	0.44	0.53	0.510	0.0650
	Adv	0.93	0.40	0.449	0.0364
	MR	0.89	0.50	0.548	0.1090
Gemini 3.1 pro-preview	Sin	0.86	0.47	0.518	0.0331
	Par	0.36	0.60	0.532	0.1191
	Adv	0.94	0.46	0.508	0.0595
	MR	0.86	0.47	0.518	0.1811
Mean F_2 over models: Sin 0.536 / Par 0.514 / Adv 0.479 / MR 0.602					

中 8 で Multi-run が最良となり (表 2), モデル選択によらず Multi-run 優位の傾向が観察された. 本結果は, 予算統制下でマルチエージェント議論 (multi-agent debate) や自己改良 (self-refinement) の優位が消失し得るという既報 [2, 4, 5] と整合的である. Multi-run 優位について, 統計的側面と指標感度の 2 面から検証する. 最良組 (GPT-5 mini/Multi-run, $F_2 = 0.826$) の項目 bootstrap 95% 信頼区間は [0.746, 0.903] で, Single 最良 (Opus 5, $F_2 = 0.738$, 信頼区間 [0.645, 0.848]) と部分的に重なるため, $N = 30$ の本実験では単一組間の有意差は示せない. 一方, モデル別集計では 12 モデル中 8 で Multi-run が最良となり, 複数モデルで同様の傾向が見られた. $\beta \in \{1, 1.5, 2, 3\}$ で F_β を振っ

表 3: F_β の β 感度分析 (構成別平均 F_β). 太字は各 β 列の最高値.

Config	F_1	$F_{1.5}$	F_2	F_3
Single	0.619	0.563	0.536	0.514
Parallel	0.424	0.477	0.514	0.556
Adversarial	0.572	0.508	0.479	0.456
Multi-run	0.665	0.623	0.603	0.585

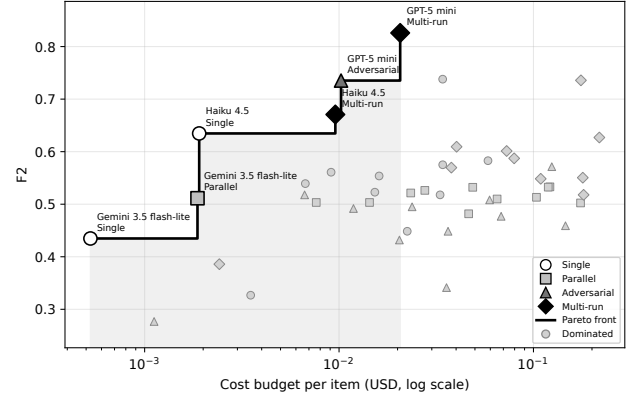


図 1: 項目あたりコスト (対数軸) と F_2 の散布. マーカの形状は構成を表す. パレートフロント上の点は黒縁で強調しモデル名を注記した (表 2 の P セルに対応). フロントから外れた点はグレーで表示する.

た感度分析 (表 3) でも Multi-run は全帯で最高値を維持する. Adversarial は再現率重み増加で順位を下げ Parallel は上げるため, $\beta = 3$ では Parallel が Single を上回る.

パレートフロントでは, 全構成で他モデルに (コスト, F_2) の両面で劣位となりフロントから外れたモデルは Sonnet 5, GPT-5, Opus 5, Fable 5, GPT-5.6 luna, GPT-5.6 terra, o3, Gemini 3.6 flash, Gemini 3.1 pro-preview である. Multi-run が単発を上回るのは項目あたり \$0.01 以上の予算帯からで, それ以上では価格帯上位モデルを単純選択するよりも下位モデルに Multi-run を組み合わせる方が支配的な戦略となる.

予算 \$0.002 では追加呼び出しの費用が支配的となり Single 構成 (Haiku 4.5 / Single) が最も高い性能を示すが, \$0.01 でオーケストレーション構成が Single を上回る. さらに \$0.05 以上では Opus 5 単体を含むすべての Single 構成を GPT-5 mini / Multi-run が上回り, 予算を 4 倍に増加させても達成可能な最高 F_2 は向上しない (図 2). 本実験の総 API 支出は \$79.79 である.

バグ種別ごとの最高 F_2 を表 4 に示す. 全 8 種別で最高値を示すのは GPT-5 mini (5 種別), Haiku 4.5 (3 種別) であり, 種別に応じて最適な (モデル, 構成) が入れ替わる. 種別別最良 F_2 は other · type-safety · api-misuse · error-handling の 4 種で 1.000 に達する一方, concurrency は最良でも 0.667 と全モデルで相

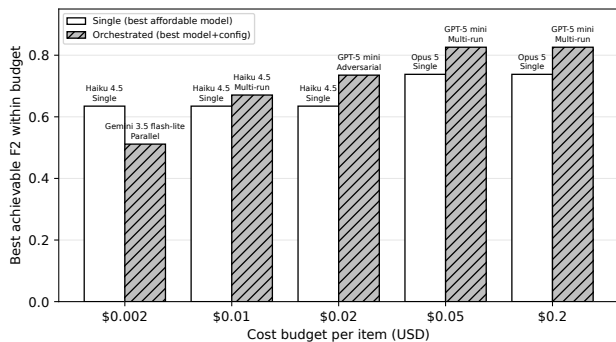


図 2: 項目あたり予算帯ごとの最高 F_2 の比較. Single 構成のみで達成可能な最高 F_2 と、オーケストレーション構成 (Parallel, Adversarial, Multi-run) で達成可能な最高 F_2 を並置する. 各棒の上には最高性能を達成したモデル・構成を注記した.

表 4: 各バグ種別における最高 F_2 とそのモデル・構成, および種別あたり項目数 N . † 付き N は小標本 ($N \leq 2$).

バグ種別	最良 (モデル / 構成)	F_2	N
logic	GPT-5 mini / Multi-run	0.779	15
type-safety	Haiku 4.5 / Single	1.000	2†
error-handling	Haiku 4.5 / Multi-run	1.000	1†
concurrency	Haiku 4.5 / Adversarial	0.667	3
api-misuse	GPT-5 mini / Multi-run	1.000	4
security	GPT-5 mini / Adversarial	0.833	1†
resource-leak	GPT-5 mini / Multi-run	0.897	2†
other	GPT-5 mini / Adversarial	1.000	2†

表 5: 各リポジトリ (言語) における最高 F_2 スコアと, そのモデル・構成.

リポジトリ (言語)	最良 (モデル / 構成)	F_2
Python (fastapi)	GPT-5 mini / Multi-run	0.722
TypeScript (valibot)	GPT-5 mini / Multi-run	0.777
Go (chi)	GPT-5 mini / Multi-run	0.972
Rust (pgrust)	GPT-5 mini / Multi-run	0.909

対的に低い値にとどまった. 種別あたりの標本数は 1 ~ 15 件と小さく個別順位は暫定的だが, RQ3 に対する予備的示唆として, 種別に応じて最適な (モデル, 構成) が入れ替わる可能性が示唆された.

言語別の最高 F_2 を表 5 に示す. 本実験の 4 リポで GPT-5 mini / Multi-run が一貫して最高値を示した. ただし対象は 4 リポに限られる. 一方で達成 F_2 は Python 0.722, TypeScript 0.777, Go 0.972, Rust 0.909 の範囲に分布し, タスクの言語・ドメインが検出難度を左右する.

6 本手法の限界

合成 PR の再現性. 逆適用 PR は実在した PR ではないため, レビュー体験の再現性に限界がある. 今

後は実在 PR サブセットとの相関を確認する.

意味一致判定の欠如. 現状のマッチングは (file, line) の重なりに基づく機械判定であり, 意味的に等価な指摘を捕捉できない場合がある. LLM 判定器 [9] はコード翻訳・生成タスクで人手評価に迫る精度を示す [10] 一方, コード評価では変数名・整形などの表層的差異による判定バイアスも報告されており [11], これらを踏まえた意味一致判定器の導入が今後の課題である.

対象モデルの限定. 本実験は主要商用 API モデル (3 ベンダ 12 モデル) に限定される. open-weight モデル (Qwen, DeepSeek 系等) はセルフホストの GPU 時間・電気代が主コストとなり本研究の API コスト統制と評価軸が異なるため, 別途統制軸を再定義した比較設計が今後の課題である.

リポジトリの偏り. pgrust は単独著者かつ AI 支援開発リポジトリのため, 一般的な OSS のバグ分布とは性質が異なる可能性がある. また対象 4 リポジトリは Web フレームワーク / スキーマ検証 / HTTP ルータ / DB エンジンに限定され, フロントエンドのコンポーネントライブラリやモバイル・ML ライブラリなど別ドメイン, および Kotlin, Swift, C++ 等の他言語は含まれない. リポジトリ数・言語・ドメインの拡張は今後の課題である.

バグ種別サンプリングの偏り. 30 項目の層別サンプリングはリポジトリ比例で行い, バグ種別で均等化していないため, logic ($N=15$) 以外の 7 種別は $N \leq 4$ で個別順位の統計的信頼性が低い (表 4 の † セルは特に小標本).

反証プロンプトの設計依存性. Adversarial の反証プロンプトは「差が明確に示さない限り REFUTED を返せ」と REFUTED 寄りに設計されており, 再現率低下の一因である可能性がある. より中立な反証設計 (例: 「支持と反証を両側から論じ, より強い側を選べ」) の下では Adversarial の性能順位が変わる余地がある.

カットオフの単一境界近似. post-cutoff 判定は 2026-01-01 の単一境界で行っており, モデルごとの真のカットオフ差 (数か月程度) は考慮していない. pgrust については全モデル共通の universal post-cutoff のため影響ゼロだが, 他 3 リポでは境界周辺で誤ラベルの余地が残る.

LLM による選定バイアス. 候補選定に LLM 分類を用いているため, 特定 LLM ファミリーが検出しやすいバグに標本が偏る余地がある. 全候補は OSS 開発者が fix: でラベルした修正コミット由来であり, バグ修正である可能性は commit 記録から支持される. 分類本体は Claude 系 (Sonnet 5) で行い, 異系統 LLM (Codex/GPT 系) の独立再判定 17 件で 12 件一致, 最終除外は 1 件 (全 85 件の 1.2%) にとどまった. 構成別 F_2 差は 0.08 レンジ (Multi-run 0.610 対 Adversarial

0.479) と 1.2% の選定変動に対して十分大きいですが、本結果を反転させる影響があったかは本実験のみからは断定できない。ただし、選定バイアス全体への厳密な感度分析は今後の課題である。また分類に用いた Claude 系モデル自体もパレートフロントから外れており (9/12), 分類側が有利となる方向のバイアスは実結果に現れていない。

7 おわりに

AI コードレビューのオーケストレーション構成をコスト-性能面で比較するためのベンチマーク構築手法を提案し、4 言語 4 リポジトリから 85 件の実バグ由来課題を構築した。30 件を対象に 4 構成 $\times$ 12 モデル = 1440 実行を実測し、同等コスト帯における構成間の性能差と、構成に依存したパレートフロントの形を確認した。今後の研究方向として、オーケストレーション特化構成の検討を進める。具体的には、RQ3 で観察されたモデル間性能差を利用した観点別役割割り当ての最適化 (バグ種別と Parallel の観点・モデル対応の設計)、Parallel と Adversarial を組み合わせて過剰指摘を反証プロンプトで抑制する複合構成、および動的ルーティング・階層集約などオーケストレーション特化構造 [3] の設計比較を検討する。残された課題として、構成非依存のシステムプロンプト最適化、サンプル拡大 (85 件全数、かつ各バグ種別 N の均等化) と扱うバグ種別の追加・細分化が挙がる。意味一致判定器としては、コード翻訳・生成タスクで人手評価と高い一致 (Pearson ≥ 0.81) が報告されている個別採点方式 [10] を導入し、位置一致と意味一致の 2 観点 $\times$ 5 段階の評価基準で採点する。コード領域の判定バイアス (変数名変更や説明文への感受性)[11] を回避するため、判定プロンプトからは PR 説明文を非表示化し、6 種の判定バイアスの事前検査を採用モデル選択に組み込む。妥当性は人手アノテータ間の一致上限 (81%)[9] を基準に、2 名アノテータの Spearman ρ で検証する。さらに Qwen や Llama 系などの open-weight LLM を第三者判定として組み合わせることで、被験モデルの Claude/OpenAI/Google 系とは独立の複数判定による一致度検証が可能となり、判定 LLM ファミリー由来のバイアスをさらに排除する余地がある。ベンチマークデータ・実験ハーネスに加え、全 1440 件の実行記録 (data/experiments/v0/*.results.jsonl) と API 呼び出し記録 (runs/*/ledger.jsonl) も配布し、第三者による追試とコスト再計算を可能にしている。

謝辞

本研究の実験に用いた API クレジットは北陸先端科学技術大学院大学 社会的信号処理・マルチモーダルインタラクション研究室の予算により提供された。

参考文献

- [1] Xunzhu Tang, Kisub Kim, Yewei Song, Cedric Lothritz, Bei Li, Saad Ezzini, Haoye Tian, Jacques Klein, and Tegawende F. Bissyande. Codeagent: Autonomous communicative agents for code review, 2024.
- [2] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. Encouraging divergent thinking in large language models through multi-agent debate. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 17889–17904, Miami, Florida, USA, November 2024. Association for Computational Linguistics.
- [3] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges, 2024.
- [4] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.
- [5] Qineng Wang, Zihao Wang, Ying Su, Hanghang Tong, and Yangqiu Song. Rethinking the bounds of llm reasoning: Are multi-agent discussions the key?, 2024.
- [6] René Just, Darioush Jalali, and Michael D. Ernst. Defects4j: a database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ISSTA 2014, page 437–440, New York, NY, USA, 2014. Association for Computing Machinery.
- [7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024.

- [8] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Yun Yang, Weisong Sun, Shengcheng Yu, and Zhenyu Chen. A survey on large language models for software engineering, 2024.
- [9] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.
- [10] Ruiqi Wang, Jiyu Guo, Cuiyun Gao, Guodong Fan, Chun Yong Chong, and Xin Xia. Can llms replace human evaluators? an empirical study of llm-as-a-judge in software engineering, 2025.
- [11] Jiwon Moon, Yerin Hwang, Dongryeol Lee, Taegwan Kang, Yongil Kim, and Kyomin Jung. Don't judge code by its cover: Exploring biases in llm judges for code evaluation, 2025.